МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО
ОБРАЗОВАНИЯ
«МОСКОВСКИЙ АВИАЦИОННЫЙ ИНСТИТУТ
(национальный исследовательский университет)»
Центр образовательных программ топ-уровня в сфере
информационных технологий Образовательного центра Института №8

ЛАБОРАТОРНАЯ РАБОТА №4

по дисциплине «Операционные системы и архитектура компьютера»

на тему: **«Обмен данными между процессами через отображаемые файлы»**

Вариант 4

Выполнил:

студент гр. М8О-205БВ-25

Гурьянов Леонид Александрович

Проверил: Филимонов Н. С.

Оценка:

Дата:

Москва 2026

Содержание

1 Цель работы **2**

2 Задание **2**

3 Описание работы программы **2**

4 Исходный код **3**

4.1 parent.c 3

4.2 child.c 6

5 Сборка и запуск **8**

6 Примеры работы **8**

7 Выводы **9**

1. Цель работы

Приобретение практических навыков в:

- освоении принципов работы с файловыми системами;
- обеспечении обмена данными между процессами посредством технологии «File mapping».

2. Задание

Составить и отладить программу на языке Си, осуществляющую работу с процессами и взаимодействие между ними. Основной процесс должен создать для решения задачи дочерний процесс. Взаимодействие между процессами осуществляется через отображаемые файлы (memory-mapped files). Необходимо обрабатывать системные ошибки, которые могут возникнуть в результате работы.

Вариант 4. Пользователь вводит команды вида «число число число». Числа передаются от родительского процесса в дочерний. Дочерний процесс делит первое число на последующие, а результат выводит в файл. Если происходит деление на 0, то дочерний и родительский процессы завершают работу. Проверка деления на 0 осуществляется на стороне дочернего процесса. Числа имеют тип `float`, их количество произвольно.

3. Описание работы программы

Родительский процесс запрашивает у пользователя имя выходного файла и строку с числами. Размер строки заранее неизвестен, поэтому она считывается посимвольно в динамический буфер.

Далее родитель создаёт обычный файл `lab4.map` функцией `open` (с флагами `O_CREAT`, `O_RDWR`, `O_TRUNC`), задаёт его размер через `ftruncate` и отображает в своё адресное пространство вызовом `mmap` с флагом `MAP_SHARED`. В начало области записывается поле статуса (4 байта), затем введённая строка вместе с завершающим нулевым символом. Размер файла вычисляется как размер заголовка плюс длина строки плюс

один байт под символ конца строки. После вызова `mmap` файловый дескриптор закрывается, так как отображение от него не зависит.

Дочерний процесс запускается через `fork` и `execl`. Так как `exec` заменяет адресное пространство процесса, отображение родителя не сохраняется, поэтому дочерний процесс получает через аргументы командной строки имя выходного файла, имя отображаемого файла и его размер, открывает тот же файл функцией `open` и отображает его у себя через `mmap`.

Дочерний процесс разбирает строку функцией `strtof`, делит первое число на каждое из последующих и записывает результат в выходной файл. Если очередной делитель равен нулю, в поле статуса записывается `ERR` и процесс завершается с кодом 1. При успехе записывается `OK`.

Родительский процесс ожидает завершения потомка через `waitpid` и читает статус из отображённой памяти. Если статус отличается от `OK`, родитель также завершается с кодом 1. В конце вызываются `munmap` и `unlink`, которые снимают отображение и удаляют служебный файл.

Системные ошибки (`open`, `ftruncate`, `mmap`, `fork`, `execl`, `fopen`) обрабатываются проверкой возвращаемых значений.

4. Исходный код

4.1. parent.c

```c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <fcntl.h>
#include <sys/mman.h>
#include <sys/wait.h>
#include <string.h>

char* NAME = "lab4";

struct shared {
    char response[4];
    char data[];
};

```

```c
void plus_char(char **str, char data, long *capacity, long *length) {
    if (*length + 2 > *capacity) {
        *capacity *= 2;
        char *new_str = (char *) realloc(*str, (*capacity) * sizeof(char));
        if (new_str == NULL) {
            exit(1);
        }
        *str = new_str;
    }
    (*str)[*length] = data;
    (*length)++;
    (*str)[*length] = '\0';
}

int main() {
    long capacity_name = 16;
    long len_name = 0;
    char *filename = (char *) malloc(capacity_name * sizeof(char));
    int c;
    long capacity_line = 16;
    long len_line = 0;
    char *line;

    if (!filename) {
        return 1;
    }

    printf("Имя файла: ");
    c = getchar();

    while (c != '\n' && c != EOF) {
        plus_char(&filename, c, &capacity_name, &len_name);
        c = getchar();
    }

    if (len_name == 0) {
        free(filename);
        return 1;
    }

    printf("Введите числа:\n");
    line = (char *) malloc(capacity_line * sizeof(char));

    if (!line) {
        free(filename);
        return 1;
    }

```

```
64      c = getchar();
65      while (c != '\n' && c != EOF) {
66          plus_char(&line, c, &capacity_line, &len_line);
67          c = getchar();
68      }
69      plus_char(&line, '\n', &capacity_line, &len_line);
70
71      size_t size = sizeof(struct shared) + len_line + 1;
72
73      int fd = open(NAME, O_CREAT | O_RDWR | O_TRUNC, 0600);
74      if (fd < 0) {
75          free(line);
76          free(filename);
77          return 1;
78      }
79      if (ftruncate(fd, size) < 0) {
80          close(fd);
81          unlink(NAME);
82          free(line);
83          free(filename);
84          return 1;
85      }
86
87      struct shared *m = mmap(NULL, size, PROT_READ | PROT_WRITE,MAP_SHARED, fd,
            0);
88      close(fd);
89      if (m == MAP_FAILED) {
90          unlink(NAME);
91          free(line);
92          free(filename);
93          return 1;
94      }
95
96      memcpy(m->data, line, len_line + 1);
97
98      char size_str[32];
99      snprintf(size_str, sizeof(size_str), "%zu", size);
100
101     pid_t pid = fork();
102
103     if (pid < 0) {
104         munmap(m, size);
105         unlink(NAME);
106         free(line);
107         free(filename);
108         return 1;
109     }
110
```

```
111     if (pid == 0) {
112         execl("./child", "child", filename, NAME, size_str, NULL);
113         exit(1);
114     }
115 
116     int status = 0;
117     waitpid(pid, &status, 0);
118 
119     int error = 0;
120     if (strncmp(m->response, "OK\n", 3) != 0) {
121         error = 1;
122     }
123 
124     munmap(m, size);
125     unlink(NAME);
126     free(line);
127     free(filename);
128 
129     return 0;
130 }
```

4.2. child.c

```
1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <unistd.h>
4  #include <fcntl.h>
5  #include <string.h>
6  #include <sys/mman.h>
7  #include <sys/stat.h>
8  
9  struct shared {
10     char response[4];
11     char data[];
12 };
13 
14 int main(int argc, char *argv[]) {
15 
16     if (argc < 4) {
17         return 1;
18     }
19 
20     char *filename = argv[1];
21     char *name = argv[2];
22     size_t shm_size = (size_t) strtoul(argv[3], NULL, 10);
23 
24     int fd = open(name, O_RDWR);
```

```
25      if (fd < 0) {
26          return 1;
27      }
28      struct shared *m = mmap(NULL, shm_size, PROT_READ | PROT_WRITE,MAP_SHARED,
            fd, 0);
29      close(fd);
30      if (m  == MAP_FAILED) {
31          return 1;
32      }
33
34      FILE *file = fopen(filename, "w");
35      if (!file) {
36          memcpy(m ->response, "ERR", 3);
37          munmap(m , shm_size);
38          return 1;
39      }
40
41      char *str = m ->data;
42      char *ptr;
43      char *endptr;
44      float first_num;
45
46      if (str[0] == '\0' || str[0] == '\n') {
47          memcpy(m ->response, "ERR", 3);
48          fclose(file);
49          munmap(m , shm_size);
50          return 1;
51      }
52
53      ptr = str;
54      first_num = strtof(ptr, &endptr);
55
56      if (ptr == endptr) {
57          memcpy(m ->response, "OK\n", 3);
58          fclose(file);
59          munmap(m , shm_size);
60          return 0;
61      }
62
63      ptr = endptr;
64      int div_by_zero = 0;
65
66      while (*ptr != '\0') {
67          float current_num = strtof(ptr, &endptr);
68
69          if (ptr == endptr) {
70              ptr++;
71              continue;
```

```
        }

        if (current_num == 0.0f) {
            div_by_zero = 1;
            break;
        }

        first_num /= current_num;
        ptr = endptr;
    }

    if (div_by_zero) {
        memcpy(m ->response, "ERR", 3);
        fclose(file);
        munmap(m , shm_size);
        exit(1);
    }

    fprintf(file, "%f\n", first_num);
    fclose(file);
    memcpy(m->response, "OK\n", 3);
    munmap(m , shm_size);

    return 0;
}
```

5. Сборка и запуск

```
gcc child.c -o child
gcc parent.c -o parent
./parent
```

Запускать `./parent` нужно из папки, где лежит исполняемый файл `child`: служебный файл `lab4.map` создаётся в текущей папке.

6. Примеры работы

Тест 1. Обычное деление.

```
Имяфайла
 : out.txtВведитечисла
 :
100 2 5
```

В файле `out.txt` записано `10.000000`.

Тест 2. Деление на ноль.

```
Имяфайла
 : out.txtВведитечисла
 :
10 0 3
```

Дочерний процесс записал `ERR` в отображаемый файл и завершился, родительский процесс тоже завершился с кодом 1.

7. Выводы

В ходе работы был реализован обмен данными между процессами через файл, отображаемый в память (`open` + `mmap`), вместо каналов, использовавшихся в лабораторной работе №2. Отображение в память позволяет процессам работать с общими данными как с обычным массивом без вызовов `read`/`write`. Режим `MAP_SHARED` обеспечивает видимость изменений для обоих процессов. Из-за использования `exec` потребовался именованный файл, который дочерний процесс открывает по имени.